Introducción: El mantenimiento a la tabla **Escuelas Profesionales** estará divido en 3 capítulos

CAPITULO I

Importante: Juntamente con el instructor analizan la tabla, los campos y tipos de datos que hay en la tabla Escuelas Profesionales

Indicaciones: Haciendo uso del programa que consideres más apropiado crear las vistas y procedimientos que a continuación se muestra, tema de base de datos ya es de conocimiento así, que en esto no nos vamos a detener tiempo ya en explicar, el alumno ya debe asumir estas responsabilidades el instructor deja como guía, lo que se requiere para el proyecto.

1. Vistas para mostrar Escuelas Profesionales (En cuanto a la vista para mostrar las facultades ya lo tenemos de la sesión anterior aremos uso para llenar los objetos jcombobox)
2. Procedure para buscar usando LIKE
3. Procedure para insertar
4. Procedure para modificar
5. Procedure para eliminar o dar de baja
6. Realizar el testeo de cada uno, si hay fallas corregir antes de pasar a la Fase II

1. Implementamos las vistas y procedimientos almacenados para la tabla Escuelas Profesionales

```
--1. Vista para mostrar EscuelaProfesional
DROP VIEW IF EXISTS v_EscuelaProfesional_Facultad;
CREATE VIEW v_EscuelaProfesional_Facultad AS
SELECT
    ep.CodigoEscuela,
    ep.NombreEscuela,
    f.NombreFacultad
FROM
    EscuelaProfesional ep
JOIN
    Facultad f ON ep.CodigoFacultad = f.CodigoFacultad
WHERE
    ep.EstadoEscuelaProfesional = 1;
```

```
--2.Procedure para buscar registros
DROP PROCEDURE IF EXISTS vera_BuscarEscuelaProfesional;
DELIMITER $$

CREATE PROCEDURE vera_BuscarEscuelaProfesional (
    IN p_parametro VARCHAR(100)
)
BEGIN
    SELECT *
    FROM v_EscuelaProfesional_Facultad
    WHERE NombreEscuela LIKE CONCAT('%', p_parametro, '%')
       OR NombreFacultad LIKE CONCAT('%', p_parametro, '%');
END$$

DELIMITER ;

CALL vera_BuscarEscuelaProfesional('ingeniería');
```

```
--3.Procedure para insertar registros
DROP PROCEDURE IF EXISTS vera_InsertarEscuelaProfesional;
DELIMITER $$
CREATE PROCEDURE vera_InsertarEscuelaProfesional (
    IN p_NombreEscuela VARCHAR(85),
    IN p_CodigoFacultad INT)
BEGIN
    -- Validar si ya existe una escuela con el mismo nombre (insensible a mayúsculas)
    IF EXISTS (
        SELECT 1 FROM EscuelaProfesional
        WHERE UPPER(NombreEscuela) = UPPER(p_NombreEscuela)
    ) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'El nombre de la escuela profesional ya está registrado.';
    END IF;
    -- Validar si la facultad existe (opcional, pero buena práctica)
    IF NOT EXISTS (
        SELECT 1 FROM Facultad WHERE CodigoFacultad = p_CodigoFacultad
    ) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'La facultad indicada no existe.';
    END IF;

    -- Insertar nuevo registro
    INSERT INTO EscuelaProfesional (NombreEscuela, CodigoFacultad)
    VALUES (p_NombreEscuela, p_CodigoFacultad);
END$$
DELIMITER ;

CALL InsertarEscuelaProfesional('Escuela de Ingeniería de Software', 1);
```

```
--4.Procedure para modificar/actualizar
DROP PROCEDURE IF EXISTS vera_ModificarEscuelaProfesional;
DELIMITER $$
CREATE PROCEDURE vera_ModificarEscuelaProfesional (
  IN p_CodigoEscuela INT,
  IN p_NombreEscuela VARCHAR(85),
  IN p_CodigoFacultad INT)
BEGIN
  -- 1. Validar que la escuela profesional exista
  IF NOT EXISTS (SELECT 1 FROM EscuelaProfesional WHERE CodigoEscuela = p_CodigoEscuela) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'La escuela profesional no existe.';
  END IF;
  -- 2. Validar que no exista otra escuela con el mismo nombre (ignorando mayúsculas/minúsculas)
  IF EXISTS (
    SELECT 1 FROM EscuelaProfesional
    WHERE LOWER(NombreEscuela) = LOWER(p_NombreEscuela)
      AND CodigoEscuela <> p_CodigoEscuela
  ) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'El nombre de la escuela ya está registrado.';
  END IF;
  -- 3. Validar que el código de facultad exista
  IF NOT EXISTS (SELECT 1 FROM Facultad WHERE CodigoFacultad = p_CodigoFacultad) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'La facultad indicada no existe.';
  END IF;
  -- 4. Actualizar la escuela
  UPDATE EscuelaProfesional
  SET NombreEscuela = p_NombreEscuela,CodigoFacultad = p_CodigoFacultad
  WHERE CodigoEscuela = p_CodigoEscuela;
END$$
DELIMITER ;
```

```
--5.Procedure para dar de baja (estado = 0)
DROP PROCEDURE IF EXISTS Vera_DarDeBajaEscuelaProfesional;
DELIMITER $$

CREATE PROCEDURE Vera_DarDeBajaEscuelaProfesional (
  IN p_CodigoEscuela INT
)
BEGIN
  -- Verificar que la escuela exista
  IF NOT EXISTS (
    SELECT 1 FROM EscuelaProfesional WHERE CodigoEscuela = p_CodigoEscuela
  ) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'La escuela no existe.';
  END IF;

  -- Actualizar estado a inactivo
  UPDATE EscuelaProfesional
  SET EstadoEscuelaProfesional = 0
  WHERE CodigoEscuela = p_CodigoEscuela;
END$$

DELIMITER ;

--ejemplo
CALL Vera_DarDeBajaEscuelaProfesional(4);
```

```sql
--6. Procedure para activar (estado = 1)
DROP PROCEDURE IF EXISTS Vera_ActivarEscuelaProfesional;
DELIMITER $$

CREATE PROCEDURE Vera_ActivarEscuelaProfesional (
  IN p_CodigoEscuela INT
)
BEGIN
  -- Verificar que la escuela exista
  IF NOT EXISTS (
    SELECT 1 FROM EscuelaProfesional WHERE CodigoEscuela = p_CodigoEscuela
  ) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'La escuela no existe.';
  END IF;

  -- Actualizar estado a activo
  UPDATE EscuelaProfesional
  SET EstadoEscuelaProfesional = 1
  WHERE CodigoEscuela = p_CodigoEscuela;
END$$

DELIMITER ;

--ejemplo
CALL Vera_ActivarEscuelaProfesional(4);
```

CAPITULO II

Nota: La clase ya lo tenemos implementado la conexión también incluso tenemos el método de mostrar escuelas profesionales después de ese método seguiremos trabando recuerda que estamos en la clase **Conexion_BD**

Indicaciones: ¿Con respecto a la clase ya hemos visto en el mantenimiento facultades, por lo tanto, ya se tiene conocimiento que tenemos implementado?, y que vamos implementar?

1. Implementar el método para llenar el objeto jcombobox
2. Implementar el método para recibir el id cuando el usuario seleccione un ítem del objeto jcombobx
3. Implementar el método listarEscuelasProfesionales
4. Implementar el método buscarEscuelasProfesionales
5. Implementar un método que valide, si el nombre de la escuela profesional ya existe insert, update
6. Implementar el método insertarEscuelasProfesionales
7. Implementar el método modificarEscuelasProfesionales
8. Implementar el método desactivarEscuelasProfesionales y activarEscuelasProfesionales

Desarrollo:

1 y 2

```
/*-----------------CRUD ESCUELA PROFESIONAL----------------------------*/
    /** Lista los nombres de todas las facultades activas (Estado = 1) usando la ...7 lines */
    public ResultSet combobox_listarFacultades() throws SQLException {
        String sql = "SELECT NombreFacultad FROM v_mostrarfacultades";
        st = conn.createStatement();
        rs = st.executeQuery(sql);
        return rs;
    }

    /** Retorna el código de una facultad dado su nombre, usando la vista ...8 lines */
    public int obtenerCodigoFacultad(String nombre) throws SQLException {
        String sql = "SELECT CodigoFacultad FROM v_mostrarfacultades WHERE NombreFacultad = ?";
        PreparedStatement ps = conn.prepareStatement(sql);
        ps.setString(1, nombre);
        ResultSet rsAux = ps.executeQuery();

        if (rsAux.next()) {
            return rsAux.getInt("CodigoFacultad");
        } else {
            return -1; // No se encontró
        }
    }
```

3 y 4

```java
/** Lista todas las escuelas profesionales activas con su facultad ...6 lines */
public ResultSet verEscuelasProfesionales() throws SQLException {
    String sql = "SELECT * FROM v_EscuelaProfesional_Facultad";
    st = conn.createStatement();
    rs = st.executeQuery(sql);
    return rs;
}
```

```java
/** Busca escuelas profesionales por nombre de escuela o facultad ...7 lines */
public ResultSet buscarEscuelaProfesional(String parametro) throws SQLException {
    String sql = "{CALL vera_BuscarEscuelaProfesional(?)}";
    CallableStatement cs = conn.prepareCall(sql);
    cs.setString(1, parametro);
    rs = cs.executeQuery();
    return rs;
}
```

5 y 6

```java
/** Valida si ya existe una escuela profesional con ese nombre ...10 lines */
public boolean existeEscuelaConNombre(String nombre, int codigoEscuela) throws SQLException {
    String sql = "SELECT 1 FROM EscuelaProfesional "
            + "WHERE LOWER(NombreEscuela) = LOWER(?) "
            + "AND CodigoEscuela <> ?";
    PreparedStatement ps = conn.prepareStatement(sql);
    ps.setString(1, nombre.trim());
    ps.setInt(2, codigoEscuela); // 0 si es insertar
    ResultSet rs = ps.executeQuery();
    return rs.next(); // si devuelve algo, ya existe
}
```

```java
/** Inserta una nueva escuela profesional si no existe una con el mismo ...9 lines */
public void insertarEscuelaProfesional(String nombre, int codigoFacultad) throws SQLException {
    // Validar si ya existe una escuela con ese nombre
    if (existeEscuelaConNombre(nombre, 0)) {
        throw new IllegalArgumentException("Ya existe una escuela con ese nombre.");
    }

    String sql = "CALL vera_InsertarEscuelaProfesional(?, ?)";
    PreparedStatement ps = conn.prepareStatement(sql);
    ps.setString(1, nombre.trim());
    ps.setInt(2, codigoFacultad);

    ps.execute();
}
```

7 y 8

```
/** Modifica los datos de una escuela profesional si no hay duplicación de ...11 lines */
public void modificarEscuelaProfesional(int codigoEscuela, String nuevoNombre,
        int nuevoCodigoFacultad) throws SQLException {
    // Validar si ya existe otra escuela con ese mismo nombre
    if (existeEscuelaConNombre(nuevoNombre, codigoEscuela)) {
throw new IllegalArgumentException("El nombre de la escuela ya está registrado en otra escuela.");
    }
    String sql = "CALL vera_ModificarEscuelaProfesional(?, ?, ?)";
    PreparedStatement ps = conn.prepareStatement(sql);
    ps.setInt(1, codigoEscuela);
    ps.setString(2, nuevoNombre.trim());
    ps.setInt(3, nuevoCodigoFacultad);

    ps.execute();
}

/** Da de baja una escuela profesional (estado = 0) ...6 lines */
public void darDeBajaEscuela(int codigoEscuela) throws SQLException {
    String sql = "{CALL Vera_DarDeBajaEscuelaProfesional(?)}";
    CallableStatement cs = conn.prepareCall(sql);
    cs.setInt(1, codigoEscuela); // Solo se pasa el código, el estado lo maneja el procedure
    cs.execute();
}
```

8.

```
/** Activa una escuela profesional (estado = 1) ...6 lines */
public void activarEscuela(int codigoEscuela) throws SQLException {
    String sql = "{CALL Vera_ActivarEscuelaProfesional(?)}";
    CallableStatement cs = conn.prepareCall(sql);
    cs.setInt(1, codigoEscuela); // Solo se pasa el código, el estado lo maneja el procedure
    cs.execute();
}
```

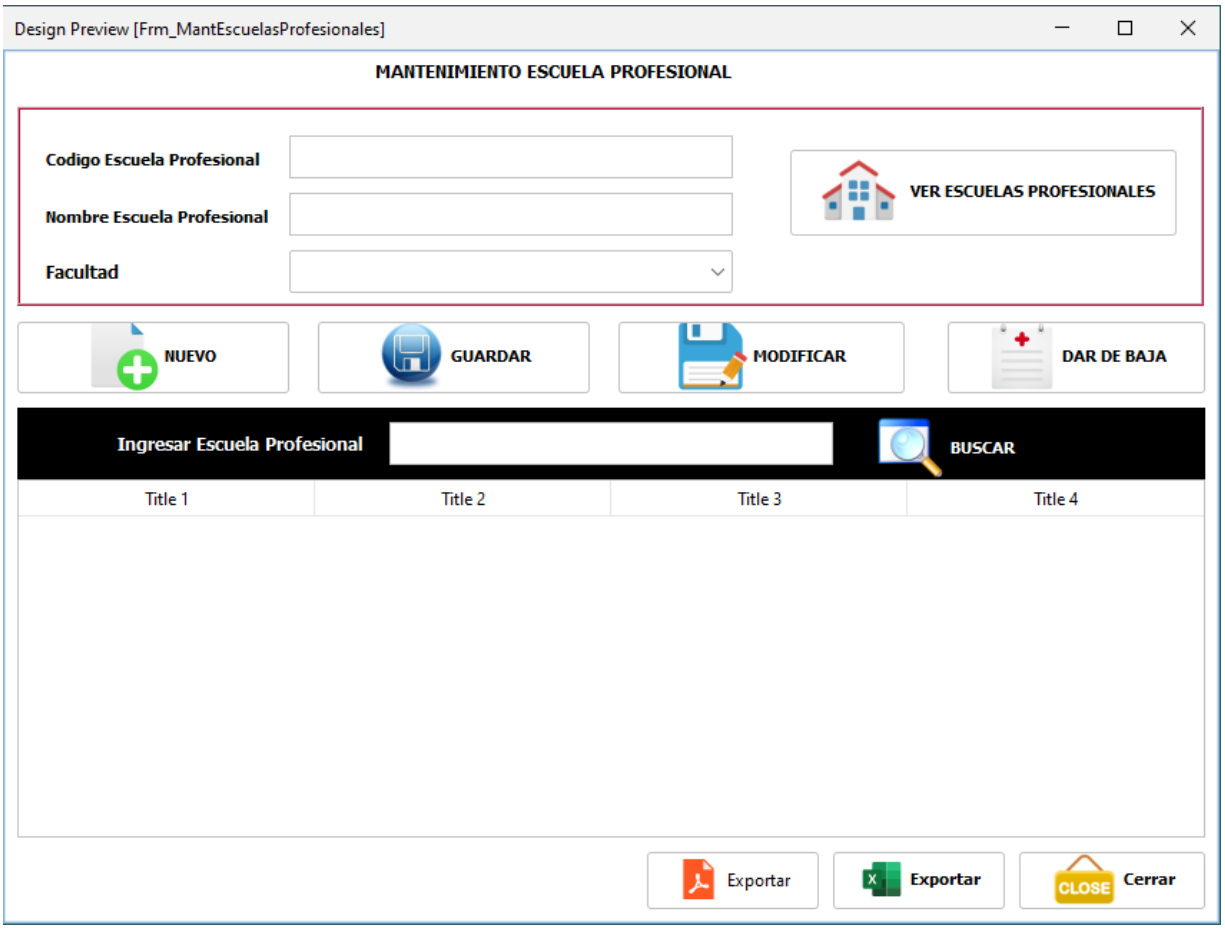
CAPITULO II

Indicaciones: Con respecto al formulario, implementaremos el diseño, según el modelo facilitado por el instructor, hay que considerar los campos que vienen de la base de datos:

1. Diseñar el formulario
2. Implementar la programación necesaria para mostrar llenar el objeto jcombobox y mostrar escuelasProfesionales
3. Implementar la programación para buscar
4. Implementar la programación para insertar
5. Implementar la programación para modificar
6. Implementar la programación para dar de baja o desactivar registro
7. Implementar la programación del botón nuevo y cerrar
8. Implementar la funcionalidad de filtro en la tabla

Desarrollo:

1.



2. Importar las librerías y la instancia de la tabla y la conexión

```java
package SistemaUniversidad;
import java.sql.*; // Librerías para conexión con MySQL
import javax.swing.table.DefaultTableModel; // Para trabajar con JTable
import javax.swing.JOptionPane;

public class Frm_MantEscuelasProfesionales extends javax.swing.JFrame {
    // Modelo para mostrar datos en la tabla
    DefaultTableModel model_escuelaprofesional = new DefaultTableModel();
    // Objeto de conexión a la base de datos (versión instanciada)
    Conexion_BD conexionBD;
    // Variables para comparar cambios en modificar
    private String nombreOriginal = "";
    private String facultadOriginal = "";

    public Frm_MantEscuelasProfesionales() {
        initComponents();

        conexionBD = new Conexion_BD(); // Inicializar la instancia global
        cargarFacultades();//método para cargar registros del jcombobox

        //Definir los encabezados de la tabla
        String titulos[] = {"Codigo Escuela", "Escuela Profesional", "Facultad"};

        //Asignar los títulos al modelo
        model_escuelaprofesional.setColumnIdentifiers(titulos);

        //Establecer el modelo a la JTable
        JTABLE_MantEscuelasProfesionales.setModel(model_escuelaprofesional);
    }
```

Nota: En la parte final crear los métodos y recuerda que estos métodos luego son llamados desde el botón que le corresponda o desde el constructor.

```java
/Método para llenar objeto del tipo jcombobox
    private void cargarFacultades() {
        try {
            jComboBoxFacultades.removeAllItems();
            jComboBoxFacultades.addItem("<<Seleccionar>>"); // Opción por defecto
            ResultSet rs = conexionBD.listarFacultades();
            while (rs.next()) {
                jComboBoxFacultades.addItem(rs.getString("NombreFacultad"));
            }
        } catch (SQLException e) {
            JOptionPane.showMessageDialog(this, "Error al cargar facultades: " + e.getMessage());
        } }
    //método para mostrar registros en el jtable
    private void listarEscuelasProfesionales() {
        try {
            // Limpiar la tabla antes de cargar nuevos datos
            model_escuelaprofesional.setRowCount(0);
            ResultSet rs = conexionBD.listarEscuelasProfesionales();
            while (rs.next()) {
                Object fila[] = {
                    rs.getInt("CodigoEscuela"),
                    rs.getString("NombreEscuela"),
                    rs.getString("NombreFacultad")
                };
                model_escuelaprofesional.addRow(fila);
            }
        } catch (SQLException e) {
    JOptionPane.showMessageDialog(this, "Error al listar escuelas profesionales: " + e.getMessage());
        }}
```

```java
private void limpiarCamposEscuela() {
    txtCodigoEscuelaProfesional.setText("");
    txtNombreEscuela.setText("");
    jComboBoxFacultades.setSelectedIndex(0); // O -1 si quieres dejarlo vacío
    BTN_GUARDAR.setEnabled(true);
    BTN_MODIFICAR.setEnabled(false);
    BTN_ELIMINAR.setEnabled(false);
    BTN_VerEscuelasProfesionales.setEnabled(false);
}
```

3. Buscar

```java
private void txtBuscarEscuelasProfesionalesKeyReleased(java.awt.event.KeyEvent evt) {
    try {
        String texto = txtBuscarEscuelasProfesionales.getText().trim();
        // Limpiar tabla antes de mostrar resultados
        model_escuelaprofesional.setRowCount(0);
        ResultSet rs = conexionBD.buscarEscuelaProfesional(texto);
        boolean hayResultados = false;
        // Llenar la tabla con los resultados
        while (rs.next()) {
            Object fila[] = {
                rs.getInt("CodigoEscuela"),
                rs.getString("NombreEscuela"),
                rs.getString("NombreFacultad")
            };
            model_escuelaprofesional.addRow(fila);
            hayResultados = true;
        }
        // Si no se encontró nada, opcionalmente mostrar mensaje
        if (!hayResultados) {
JOptionPane.showMessageDialog(this, "No se encontraron escuelas profesionales con ese criterio.",
        "Sin resultados", JOptionPane.INFORMATION_MESSAGE);
        }

    } catch (SQLException e) {
        JOptionPane.showMessageDialog(this, "Error al buscar escuelas: " + e.getMessage(),
                "Error", JOptionPane.ERROR_MESSAGE);
    }

}
```

4. Botón Guardar

```java
private void BTN_GUARDARActionPerformed(java.awt.event.ActionEvent evt) {
    try {
        // 1. Obtener datos del formulario
        String nombreEscuela = txtNombreEscuela.getText().trim();
        String nombreFacultad = (String) jComboBoxFacultades.getSelectedItem();

        // 2. Validar campos requeridos
        if (nombreEscuela.isEmpty()) {
    JOptionPane.showMessageDialog(this, "Ingrese el nombre de la escuela profesional.",
            "Validación", JOptionPane.WARNING_MESSAGE);
            return;
        }

        if (nombreFacultad == null || nombreFacultad.trim().isEmpty()) {
    JOptionPane.showMessageDialog(this, "Seleccione una facultad válida.", "Validación",
            JOptionPane.WARNING_MESSAGE);
            return;
        }

        // 3. Obtener el código de la facultad desde la vista
        int codigoFacultad = conexionBD.obtenerCodigoFacultad(nombreFacultad);
        if (codigoFacultad == -1) {
    JOptionPane.showMessageDialog(this, "No se encontró la facultad seleccionada.", "Error",
            JOptionPane.ERROR_MESSAGE);
            return;
        }
```

```java
        // 4. Insertar usando método de clase (que valida duplicados)
        conexionBD.insertarEscuelaProfesional(nombreEscuela, codigoFacultad);

        // 5. Confirmar éxito
    JOptionPane.showMessageDialog(this, "✅ Escuela profesional registrada correctamente.",
            "Éxito", JOptionPane.INFORMATION_MESSAGE);

        // 6. Refrescar tabla
        listarEscuelasProfesionales();

        // 7. Limpiar campos
        this.limpiarCamposEscuela();

    } catch (IllegalArgumentException ex) {
    JOptionPane.showMessageDialog(this, ex.getMessage(), "Validación",
            JOptionPane.WARNING_MESSAGE);
    } catch (SQLException ex) {
    JOptionPane.showMessageDialog(this, "Error al guardar: " + ex.getMessage(), "Error",
            JOptionPane.ERROR_MESSAGE);
    }

}
```

5. Modificar: Evento de la tabla, para enviar el valor de la fila seleccionada a las cajas de texto.

```java
private void JTABLE_ESCUELASPROFMouseClicked(java.awt.event.MouseEvent evt) {
int filaSeleccionada = JTABLE_ESCUELASPROF.getSelectedRow();
if (filaSeleccionada != -1) {
    // Obtener los datos de la fila seleccionada
String codigo = JTABLE_ESCUELASPROF.getValueAt(filaSeleccionada, 0).toString();
String nombreEscuela = JTABLE_ESCUELASPROF.getValueAt(filaSeleccionada, 1).toString().trim();
String nombreFacultad = JTABLE_ESCUELASPROF.getValueAt(filaSeleccionada, 2).toString().trim();
    // Mostrar en los controles
    txtCodigoEscuelaProfesional.setText(codigo);
    txtNombreEscuela.setText(nombreEscuela);
    // Guardar valores originales para comparación
    nombreOriginal = nombreEscuela;
    facultadOriginal = nombreFacultad;
    // Buscar coincidencia en el ComboBox ignorando mayúsculas
    boolean encontrado = false;
    for (int i = 0; i < jComboBoxFacultades.getItemCount(); i++) {
        String item = jComboBoxFacultades.getItemAt(i).trim();
        if (item.equalsIgnoreCase(nombreFacultad)) {
            jComboBoxFacultades.setSelectedIndex(i);
            encontrado = true;
            break;
        }}
    // Si no se encontró, mostrar alerta opcional
    if (!encontrado) {
        JOptionPane.showMessageDialog(this,"No se encontró la facultad en la lista del combo.",
                "Facultad no encontrada", JOptionPane.WARNING_MESSAGE);
    }}
}
```

Botón Modificar

```
private void BTN_MODIFICARActionPerformed(java.awt.event.ActionEvent evt) {
    try {
    // Obtener datos del formulario
    String codigoStr = txtCodigoEscuelaProfesional.getText().trim();
    String nuevoNombreEscuela = txtNombreEscuela.getText().trim();
    String nombreFacultad = (String) jComboBoxFacultades.getSelectedItem();

    // Validar campos obligatorios
    if (codigoStr.isEmpty() || nuevoNombreEscuela.isEmpty() ||
            nombreFacultad == null || nombreFacultad.isEmpty()) {
   JOptionPane.showMessageDialog(this, "Complete todos los campos obligatorios.",
                "Validación", JOptionPane.WARNING_MESSAGE);
        return;
    }
    // Validar longitud del nombre
    if (nuevoNombreEscuela.length() > 85) {
JOptionPane.showMessageDialog(this, "El nombre de la escuela no debe exceder 85 caracteres.",
        "Validación", JOptionPane.WARNING_MESSAGE);
        return;
    }
    // Validar si hubo cambios
if (nuevoNombreEscuela.equalsIgnoreCase(nombreOriginal) && nombreFacultad.equalsIgnoreCase
    (facultadOriginal)) {
JOptionPane.showMessageDialog(this, "No se detectaron cambios en los datos.",
        "Información", JOptionPane.INFORMATION_MESSAGE);
        return;
    }
```

```
    // Convertir código a entero
    int codigoEscuela = Integer.parseInt(codigoStr);
    // Obtener código de la facultad
    int codigoFacultad = conexionBD.obtenerCodigoFacultad(nombreFacultad);
    if (codigoFacultad == -1) {
    JOptionPane.showMessageDialog(this, "La facultad seleccionada no es válida.",
                "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }
    // Verificar duplicidad de nombre con otras escuelas
    if (conexionBD.existeEscuelaConNombre(nuevoNombreEscuela, codigoEscuela)) {
    JOptionPane.showMessageDialog(this, "Ya existe otra escuela con el mismo nombre.",
                "Validación", JOptionPane.WARNING_MESSAGE);
        return;
    }
    // Llamar al método de modificación
    conexionBD.modificarEscuelaProfesional(codigoEscuela, nuevoNombreEscuela, codigoFacultad);
    JOptionPane.showMessageDialog(this, "Escuela profesional actualizada correctamente.",
            "Éxito", JOptionPane.INFORMATION_MESSAGE);
    // Refrescar tabla y limpiar
    listarEscuelasProfesionales();
    limpiarCamposEscuela();
} catch (NumberFormatException e) {
    JOptionPane.showMessageDialog(this, "Código de escuela no válido.", "Error", JOptionPane.ERROR_MESSAGE);
} catch (SQLException e) {
    JOptionPane.showMessageDialog(this, "Error al modificar: " + e.getMessage(), "Error",
            JOptionPane.ERROR_MESSAGE);
}
```

6. Botón dar de baja o Eliminar o Cambiar el estado del valor en la base de datos

```
private void BTN_ELIMINARActionPerformed(java.awt.event.ActionEvent evt) {
    try {
        // Obtener el código de escuela
        String codigoStr = txtCodigoEscuelaProfesional.getText().trim();
        // Validar que se haya seleccionado una fila
        if (codigoStr.isEmpty()) {
            JOptionPane.showMessageDialog(this, "Seleccione una escuela de la tabla para dar de baja.",
                    "Validación", JOptionPane.WARNING_MESSAGE);
            return;
        }
        int codigoEscuela = Integer.parseInt(codigoStr);
        // Confirmar con el usuario
        int confirmacion = JOptionPane.showConfirmDialog(this,
         "¿Está seguro de dar de baja esta escuela profesional?", "Confirmar acción",
                JOptionPane.YES_NO_OPTION);
        if (confirmacion == JOptionPane.YES_OPTION) {
            // Llamar al método de la clase para cambiar estado
            conexionBD.darDeBajaEscuela(codigoEscuela);
            JOptionPane.showMessageDialog(this, "Escuela profesional dada de baja correctamente.",
                    "Éxito", JOptionPane.INFORMATION_MESSAGE);
            // Actualizar la tabla y limpiar los controles
            listarEscuelasProfesionales(); limpiarCamposEscuela();
        }
    } catch (NumberFormatException e) {
        JOptionPane.showMessageDialog(this, "Código de escuela inválido.",
                "Error", JOptionPane.ERROR_MESSAGE);
    } catch (SQLException e) {
        JOptionPane.showMessageDialog(this, "Error al dar de baja: " + e.getMessage(),
                "Error en base de datos", JOptionPane.ERROR_MESSAGE);}
```

7. Botón Cerrar

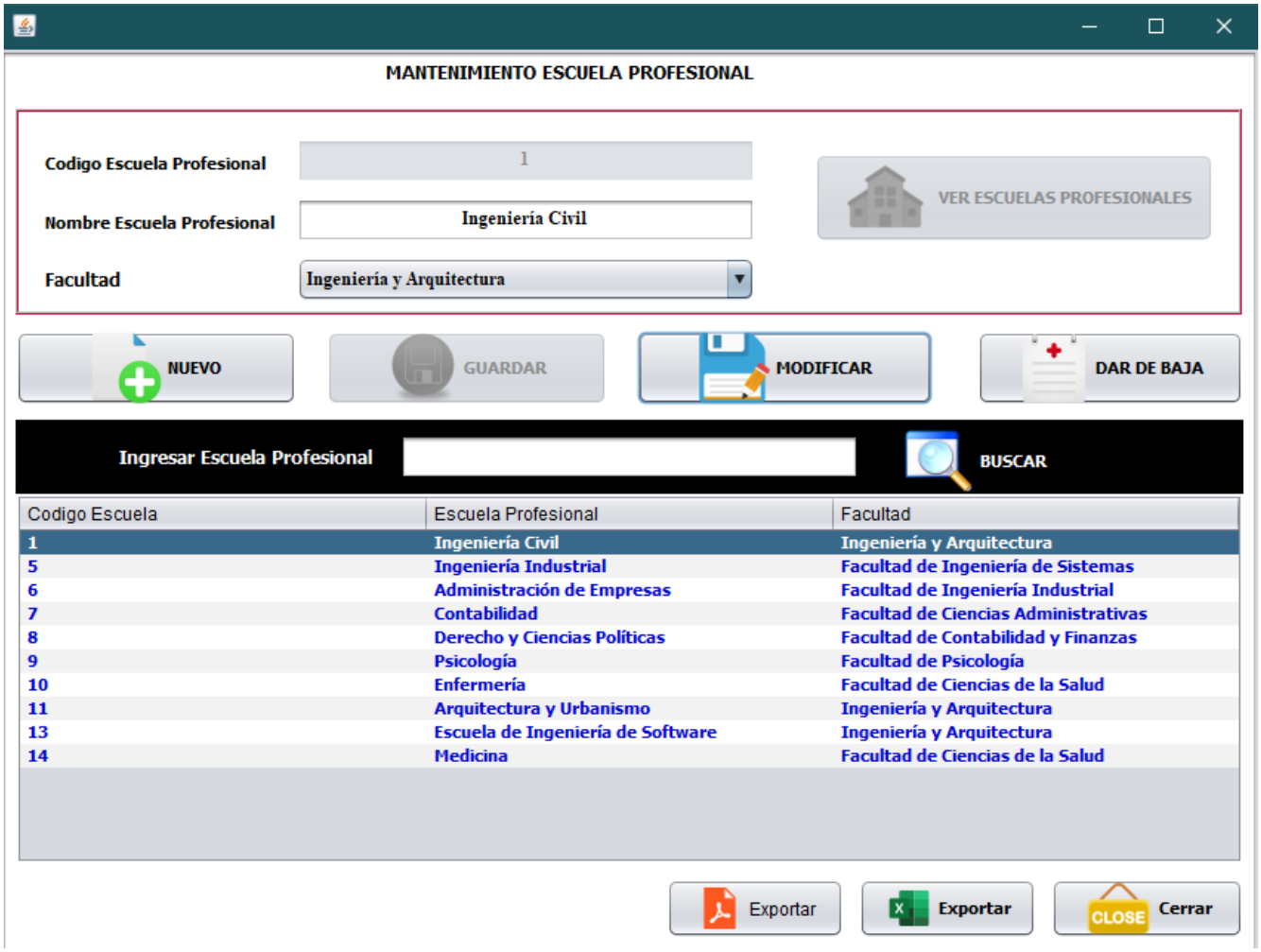
```
private void BTN_Cerrar1ActionPerformed(java.awt.event.ActionEvent evt) {
    int confirmacion = JOptionPane.showConfirmDialog(this,
            "¿Estás seguro de que deseas cerrar el formulario?",
            "Confirmar salida",
            JOptionPane.YES_NO_OPTION);

    if (confirmacion == JOptionPane.YES_OPTION) {
        try {
            // Cerrar conexión si tienes un método cerrarConexion()
            conexionBD.cerrarConexion();
        } catch (Exception e) {
            System.err.println("Error al cerrar la conexión: " + e.getMessage());
        }

        // Cierra el formulario actual
        dispose();  // o this.dispose() si estás dentro del formulario
    }

}
```

RESULTADO



Comentario: Se ha realizado el proceso CRUD para la tabla Escuelas Profesionales de la base de datos Universidad, teniendo como tabla maestra a la tabla Facultad.